STAGE 1 · INFRASTRUCTURE & INTEGRATION

Reusable Marketing Site Template

Step-by-step setup guide for a headless **WordPress** backend, an **Astro** frontend, and **Vercel** deployment.

Astro 7 | React 19 | WordPress 6.7+ | WPGraphQL | Vercel

AUDIENCE
Junior full-stack / frontend developers

DOCUMENT ID
SOP-TEMPLATE-STAGE-1

VERSION
1.0.0 — verify package versions against current releases

REFERENCE BUILD
Good Rep Media (goodrep-media-frontend)

Includes: setup report · WordPress backend · frontend project · Vercel · starter layout · verification checklist · troubleshooting · Stage 2 handoff

How to use this guide

Work through the sections in order. Each step explains *why* it matters, so you can troubleshoot instead of just copying. Look for these markers:

DECISION NEEDED A choice the team must confirm. A recommended default is given.

ASSUMPTION Something we assumed. Confirm it before relying on it.

- [ ] A checklist item you tick off once it is verified.

The build at a glance

1. **Overview** — Architecture, versions, env vars
2. **WordPress** — Install, plugins, MU-plugin
3. **Frontend** — Astro project + GraphQL client
4. **Vercel** — Link repo, env vars, DNS
5. **Layout** — Header, footer, pages, form
6. **Verify** — Checklist before sign-off
7. **Fix** — 10 common errors
8. **Stage 2** — Inputs for the CLI file

Contents

1 **Setup Report (Overview)**
- 1.1 System Architecture Diagram
- 1.2 Core Jargon Glossary (Read First)
- 1.3 Rationale for Technology Choices & Version Matrix
- 1.4 Prerequisites & Required Accounts
- 1.5 Environment Variables Matrix

2 **WordPress Backend Setup**
- 2.1 Hosting & Local Environment Recommendation
- 2.2 WordPress Installation & Subdomain Strategy
- 2.3 Required Plugins & Exact Activation Order
- 2.4 Must-Use (MU) Plugin PHP Snippets (CORS, Limits, Custom Fields & Webhooks)
- 2.5 Permalink, REST & GraphQL Endpoint Verification
- 2.6 Critical WordPress Settings Often Missed

3 **Frontend Project Setup**
- 3.1 Project Creation Commands
- 3.2 Exact Dependency Installation & `package.json`
- 3.3 Folder Structure Explanation
- 3.4 Connecting Astro to WPGraphQL (`src/lib/api.js`)

4 **Vercel Integration**
- 4.1 Linking the Git Repository to Vercel
- 4.2 Build Settings, Output Directory & Edge Config (`vercel.json`)
- 4.3 Setting Environment Variables in Vercel
- 4.4 Connecting Custom / Staging Domains via WHM & cPanel DNS
- 4.5 Verifying Preview & Production Deployments

5 **Basic Template Layout (Copy-Pasteable Starter Code)**
- 5.1 Site Configuration (`src/config/site.js`)
- 5.2 Head & Analytics Components (`BaseHead.astro` & `TrackingHead.astro`)
- 5.3 Starter Components (`Header.astro` , `Footer.astro` , `ContactForm.jsx`)
- 5.4 Master Layout Wrapper (`src/layouts/MainLayout.astro`)
- 5.5 Core Pages (`index.astro` , `about.astro` , `contact.astro` , `thankyou.astro`)

6 **Verification Checklist**

7 **Troubleshooting Guide**

8 **Handoff to Stage 2**

1 Setup Report (Overview)

1.1 System Architecture Diagram

In a **headless (decoupled) architecture**, the backend where editors write content (WordPress) is completely separated from the frontend website that visitors see (Astro on Vercel).

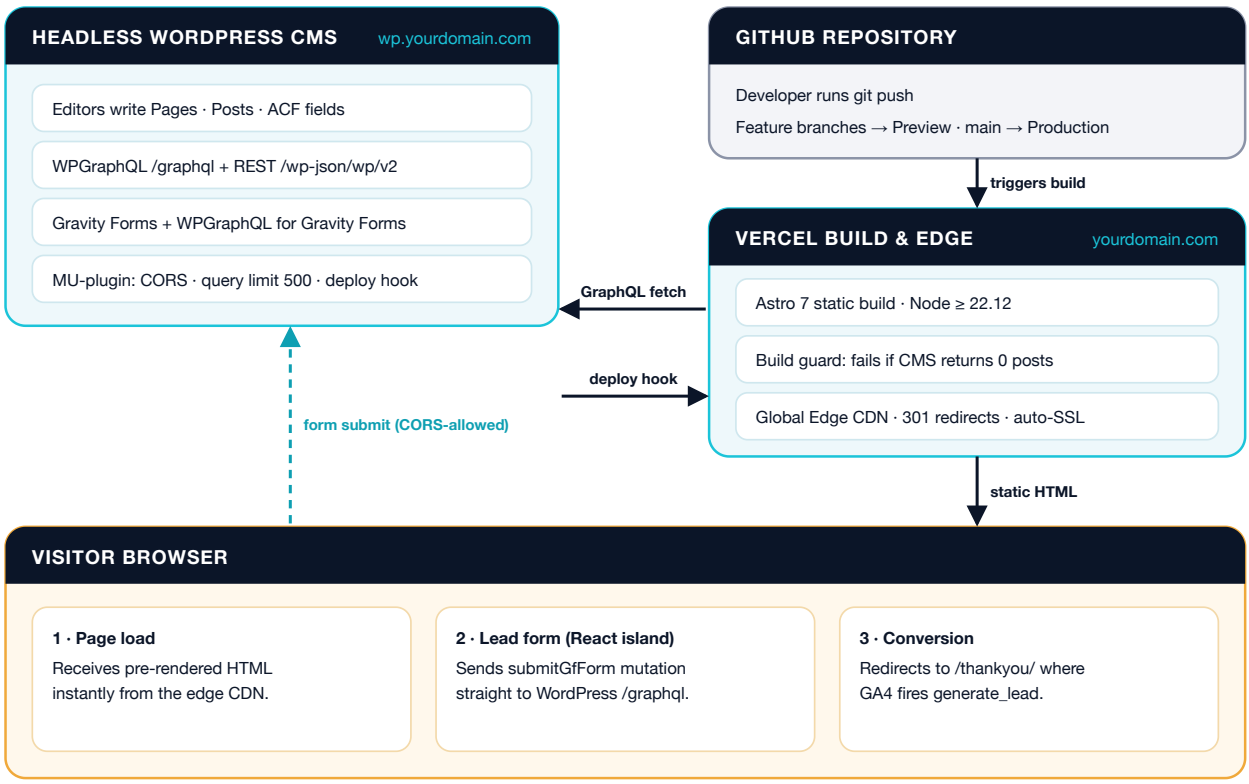


Figure 1 · Editors publish in WordPress, Vercel rebuilds the static Astro site, and visitors get fast pre-rendered pages. Only the lead form talks to WordPress at runtime.

1.2 Core Jargon Glossary (Read First)

- **Headless CMS (Content Management System):** Using WordPress purely as a database and admin dashboard (`/wp-admin`). WordPress does not render themes for public visitors; instead, it sends raw data to our frontend.
- **SSG (Static Site Generation):** A build process where Astro fetches all WordPress content once during deployment and compiles plain `.html` files. Visitors receive pre-built HTML from a global server in milliseconds rather than waiting for PHP and MySQL to build pages on the fly.

- **GraphQL (`WPGraphQL`):** A query language that lets the frontend request *exact* fields (e.g., `title` , `slug` , `heroHeadline`) in a single network request, rather than downloading huge JSON payloads.
- **Islands Architecture (Astro Islands):** Astro renders 100% static HTML with zero JavaScript by default. When a specific widget needs interactivity (like a React Contact Form), Astro loads JavaScript *only* for that isolated component ("island").
- **CORS (Cross-Origin Resource Sharing):** A browser security feature that blocks `https://yourdomain.com` from sending form data to `https://wp.yourdomain.com` unless the WordPress server explicitly permits it via HTTP headers.
- **MU-Plugin (Must-Use Plugin):** A special PHP file placed in `wp-content/mu-plugins/` that WordPress loads automatically on every request. Unlike regular plugins, it cannot be accidentally deactivated by an editor in the WordPress dashboard.
- **Build Guard:** A safety check in our frontend code that intentionally fails the Vercel build if WordPress returns 0 posts or an error, ensuring a broken build never overwrites your live production website.

1.3 Rationale for Technology Choices & Version Matrix

FOR TEAM CONFIRMATION

DECISION NEEDED **Frontend Framework: Astro vs. Next.js**

Recommended Default: Astro (`^7.3.8`)

Why: Based on our production Good Rep Media implementation (`package.json`), Astro ships zero client-side JavaScript by default, achieves near-100 Mobile PageSpeed Insights scores out of the box, compiles 200+ static pages in ~8 seconds, and still lets us mount React 19 components (`@astrojs/react`) for interactive forms.

LAYER	TECHNOLOGY / PACKAGE	REQUIRED VERSION	RATIONALE & PURPOSE
Runtime	Node.js	`>= 22.12.0` (LTS)	Required by Astro 7 for native ES module support and fast image optimization.
Frontend Framework	`astro`	`^7.3.8`	Static Site Generator (SSG) with built-in `<ClientRouter />` and `astro:assets` .
Interactive UI Islands	`react` & `react-dom`	`^19.3.1`	Powers interactive client-side components like `ContactForm.jsx` / `LeadForm.jsx` .
Astro React Bridge	`@astrojs/react`	`^7.0.1`	Allows Astro `.astro` pages to mount and hydrate `.jsx` React components.
SEO Sitemap	`@astrojs/sitemap`	`^3.7.4`	Automatically generates `sitemap-index.xml` and `sitemap-0.xml` on every build.

LAYER	TECHNOLOGY / PACKAGE	REQUIRED VERSION	RATIONALE & PURPOSE
HTML Sanitization	`isomorphic-dompurify`	`^3.19.0`	Scrubs WordPress HTML and form confirmation messages to prevent XSS attacks.
Self-Hosted Fonts	`@fontsource/poppins`, `@fontsource/open-sans`	`^5.3.0`	Bundles WOFF2 fonts locally so the browser makes zero blocking calls to Google Fonts.
Icons	`material-symbols`	`^0.40.2`	Google Material Symbols icon font (subsetted to 85 KiB in production).
Animation	`gsap`	`^3.15.0`	Handles scroll reveals and animations without layout shift.
CMS Backend	WordPress + PHP	`WP >= 6.7` / `PHP >= 8.2`	Industry-standard editorial experience with fast PHP 8.2+ execution for GraphQL.
Database	MySQL / MariaDB	`MySQL >= 8.0` / `MariaDB >= 10.6`	Relational database backing WordPress content, custom fields, and form entries.

Note: Always verify package versions against current releases using `npm outdated` before starting a new client project.

1.4 Prerequisites & Required Accounts

Before starting, ensure the junior developer has the following installed locally and has access to the required accounts.

Local software requirements

1. **Node.js `>= 22.12.0`:** Verify by running `node -v` in your terminal. (Recommendation: use `nvm install 22` to manage Node versions.)
2. **npm `>= 10.x`:** Installed automatically with Node.js. Verify with `npm -v`.
3. **Git `>= 2.39+`:** Verify with `git --version`.
4. **LocalWP (by Flywheel)** *(for local WordPress development)* OR access to the agency's staging cPanel/WHM server (`PHP 8.2+`, `MySQL 8.0+` / `MariaDB 10.6+`).
5. **Code editor:** VS Code or Antigravity IDE with the official **Astro** extension installed.

Required accounts & credentials

1. **GitHub account** with read/write access to the agency's GitHub organization.
2. **Vercel account** connected to the agency's GitHub organization.
3. **WordPress administrator login:** access to `/wp-admin` on the target WordPress instance.
4. **cPanel / WHM access (or DNS provider):** access to **Zone Editor** to point domains to Vercel and `wp.<domain.com>` to the WordPress server IP.

5. **Gravity Forms license key** *(if using Gravity Forms for lead capture)*.
   ASSUMPTION: Agency holds an Elite/Developer license
6. **Google Analytics 4 (GA4) Measurement ID:** format `G-XXXXXXXXXX` (each client site must use its own isolated GA4 property stream).

1.5 Environment Variables Matrix

Environment variables keep URLs, API endpoints, and tracking IDs out of hardcoded source files so the same codebase works across local development, staging, and production.

> **REMEMBER**
>
> **Important Astro rule:** Any environment variable prefixed with `PUBLIC_` is readable in both server-side build code **and** client-side browser code (like React forms). Never put secret private keys inside a `PUBLIC_` variable!

VARIABLE NAME	EXAMPLE VALUE	WHERE IT IS SET	PURPOSE
`PUBLIC_SITE_URL`	`https://goodrep.media`	Local `.env` & Vercel Dashboard	Canonical production domain used for `<link rel="canonical">`, OpenGraph, and JSON-LD schema.
`PUBLIC_WORDPRESS_API_URL`	`https://wp.goodrep.media/graphql` *(or `/wp-json/wp/v2`)*	Local `.env` & Vercel Dashboard	Primary endpoint used by `src/lib/api.js` during build time to fetch pages, posts, and categories.
`PUBLIC_WORDPRESS_GRAPHQL_URL`	`https://wp.goodrep.media/graphql`	Local `.env` & Vercel Dashboard	Explicit GraphQL endpoint used by client-side React forms for `submitGfForm` mutations.
`PUBLIC_GA_MEASUREMENT_ID`	`G-JP63DEKVVF`	Local `.env` & Vercel Dashboard	Dedicated GA4 Measurement ID for the specific client property (prevents cross-client analytics pollution).
`GITHUB_PAGES`	`true` *(or omitted)*	GitHub Actions CI only	When `"true"`, injects `<meta name="robots" content="noindex,follow">` so staging builds are never indexed.

2 WordPress Backend Setup

2.1 Hosting & Local Environment Recommendation

FOR TEAM CONFIRMATION

DECISION NEEDED **Local development vs. remote staging WordPress**

Recommended Default: Use a **remote staging subdomain (`https://wp.<domain.com>`)** on our managed WHM/cPanel LiteSpeed server for team collaboration, OR **LocalWP** (localwp.com) for offline sandbox testing.

Why remote staging (`wp.<domain.com>`) is preferred for agency templates: content writers and developers can work simultaneously without syncing local SQL databases, and Vercel cloud builds can immediately reach the WordPress API over HTTPS.

If using LocalWP locally:

1. Download and open **LocalWP**.
2. Click **+ Create a new site** and name it `<brand>-cms`.
3. Select **Custom Environment**: **PHP** `8.2.x`, **Web Server** `nginx` **or** `Apache`, **Database** `MySQL 8.0`.
4. Set your admin username and password, then click **Add Site**.
5. Click **Trust** next to SSL in LocalWP so your local site runs over `https://<brand>-cms.local`.

2.2 WordPress Installation & Subdomain Strategy

When building a decoupled marketing site, `https://<domain.com>` will point to **Vercel**, while WordPress lives permanently on `https://wp.<domain.com>`.

Step A: Create the `wp.` subdomain on your WordPress host (cPanel)

1. Log into **cPanel** on your WordPress hosting server.
2. Go to **Domains → Create A New Domain**.
3. Enter `wp.<domain.com>` (e.g., `wp.goodrep.media`). Uncheck "Share document root" so it gets its own clean directory (e.g., `/home/<user>/wp.<domain.com>`).
4. Ensure an **A record** exists in **cPanel → Zone Editor** for `wp.<domain.com>` pointing to your WordPress server's IPv4 address (e.g., `23.111.181.43`).

Step B: Install WordPress

1. Open **cPanel → Softaculous / WordPress Manager** (or install manually).

2. Select protocol `https://` and domain `wp.<domain.com>`. Leave the "In Directory" field **blank**.
3. Set the admin username (never use `admin`), a strong password, and an agency admin email.
4. Click **Install**.

2.3 Required Plugins & Exact Activation Order

Log in to `https://wp.<domain.com>/wp-admin/plugins.php`. Install and activate the following plugins in this **exact order** (activating base plugins before their bridge extensions prevents fatal dependency errors):

ORDER	PLUGIN NAME	SLUG / SOURCE	WHY IT IS REQUIRED
1	**WPGraphQL**	`wp-graphql` (WordPress.org)	Exposes the `/graphql` API endpoint so Astro can query pages, posts, categories, and media in a single request.
2	**Advanced Custom Fields (ACF)**	`advanced-custom-fields` (WP.org or PRO)	Lets us create structured fields (e.g., Hero Headline, Hero Subheadline) on the Home and About pages instead of messy freeform HTML.
3	**WPGraphQL for ACF**	`wpgraphql-acf` (WordPress.org)	Automatically exposes ACF field groups into the WPGraphQL schema when "Show in GraphQL" is enabled.
4	**Yoast SEO**	`wordpress-seo` (WordPress.org)	Manages page/post SEO titles, meta descriptions, and OpenGraph images, and exposes `yoast_head_json` in REST.
5	**Add WPGraphQL SEO**	`add-wpgraphql-seo` (WordPress.org)	Bridges Yoast SEO into WPGraphQL so queries can fetch `seo { title metaDesc }`.
6	**Gravity Forms** *(optional / recommended)*	`gravityforms` (commercial ZIP)	Stores form submissions in the database and sends admin email notifications. **ASSUMPTION: used for production forms as in Good Rep Media**
7	**WPGraphQL for Gravity Forms**	`wp-graphql-gravity-forms` (GitHub release)	Exposes the `submitGfForm` GraphQL mutation so React forms can submit entries directly from the browser.

2.4 Must-Use (MU) Plugin PHP Snippets (CORS, Limits, Custom Fields & Webhooks)

Why use an MU-plugin instead of `functions.php`? If someone switches the active WordPress theme inside `/wp-admin`, everything in `functions.php` disappears, instantly breaking CORS, custom fields, and Vercel webhooks. Placing code in `wp-content/mu-plugins/` guarantees it runs permanently.

Step 1: Create the file. Using FTP, SSH, or **cPanel → File Manager**, navigate to `/wp-content/` and create a folder named `mu-plugins` (if it does not already exist). Inside it, create a file named `headless-cms-config.php`:

wp-content/mu-plugins/headless-cms-config.php PATH / STRUCTURE

```
wp-content/mu-plugins/headless-cms-config.php
```

Step 2: Paste the complete PHP configuration. Copy and paste the exact code below into that file:

PHP

```php
<?php
/**
 * Plugin Name: Universal Headless WordPress Configuration
 * Description: Configures CORS headers, WPGraphQL query limits, ACF/Page GraphQL fields, and Vercel
deploy webhooks.
 * Version: 1.0.0
 * Author: Engineering Team
 */

if ( ! defined( 'ABSPATH' ) ) {
    exit;
}

/**
 * 1. INCREASE WPGRAPHQL MAX QUERY LIMIT
 * Why: By default, WPGraphQL caps queries at 100 items. Raising this to 500 allows Astro
 * to fetch all blog posts and pages in a single build-time request.
 */
add_filter( 'graphql_connection_max_query_amount', function( $amount, $source, $args, $context, $info ) {
    return 500;
}, 10, 5 );

/**
 * 2. CONFIGURE CORS HEADERS FOR FRONTEND & VERCEL PREVIEW DOMAINS
 * Why: Allows browser-based React forms (ContactForm / LeadForm) on your frontend domain
 * and Vercel preview URLs to send POST requests to /graphql and /wp-json.
 */
add_action( 'init', function() {
    $allowed_origins = [
        'https://goodrep.media',       // Replace with production domain: https://<domain.com>
        'https://www.goodrep.media',   // Replace with www domain: https://www.<domain.com>
        'http://localhost:4321',       // Astro local dev server
        'http://localhost:3000',       // Alternative local dev port
    ];

    $origin = isset( $_SERVER['HTTP_ORIGIN'] ) ? $_SERVER['HTTP_ORIGIN'] : '';

    // Also allow any *.vercel.app preview deployment URL for this team
    $is_vercel_preview = (bool) preg_match( '/^https:\/\/[a-z0-9-]+\.vercel\.app$/i', $origin );

    if ( in_array( $origin, $allowed_origins, true ) || $is_vercel_preview ) {
        header( 'Access-Control-Allow-Origin: ' . $origin );
        header( 'Access-Control-Allow-Methods: POST, GET, OPTIONS' );
        header( 'Access-Control-Allow-Headers: Content-Type, Authorization, X-WP-Nonce' );
        header( 'Access-Control-Allow-Credentials: true' );
    }

    // Immediately resolve preflight OPTIONS requests with HTTP 200
```

```
    if ( isset( $_SERVER['REQUEST_METHOD'] ) && $_SERVER['REQUEST_METHOD'] === 'OPTIONS' ) {
        status_header( 200 );
        exit;
    }
} );

/**
 * 3. REGISTER STRUCTURED ACF FIELDS FOR HOME & ABOUT PAGES IN CODE
 * Why: Registering these fields via PHP guarantees every new site launched from this
 * template immediately has the exact GraphQL schema expected by index.astro and about.astro,
 * even before an editor touches the ACF GUI.
 */
add_action( 'acf/init', function() {
    if ( ! function_exists( 'acf_add_local_field_group' ) ) {
        return;
    }

    acf_add_local_field_group( [
        'key'                   => 'group_template_page_sections',
        'title'                 => 'Template Page Content (Headless)',
        'show_in_graphql'       => 1,
        'graphql_field_name'    => 'templatePageFields',
        'map_graphql_types_from_location_rules' => 0,
        'fields'                => [
            [
                'key'               => 'field_hero_kicker',
                'label'             => 'Hero Kicker (Small Eyebrow Text)',
                'name'              => 'heroKicker',
                'type'              => 'text',
                'show_in_graphql'   => 1,
                'default_value'     => 'GROWTH MARKETING AGENCY',
            ],
            [
                'key'               => 'field_hero_headline',
                'label'             => 'Hero Headline (H1)',
                'name'              => 'heroHeadline',
                'type'              => 'text',
                'show_in_graphql'   => 1,
                'default_value'     => 'Marketing Engineered for Measurable Revenue',
            ],
            [
                'key'               => 'field_hero_subheadline',
                'label'             => 'Hero Subheadline (Lead Paragraph)',
                'name'              => 'heroSubheadline',
                'type'              => 'textarea',
                'show_in_graphql'   => 1,
                'default_value'     => 'We build ultra-fast decoupled websites and ROI-driven campaigns
for growing brands.',
            ],
            [
                'key'               => 'field_primary_cta_label',
                'label'             => 'Primary CTA Button Label',
                'name'              => 'primaryCtaLabel',
                'type'              => 'text',
                'show_in_graphql'   => 1,
                'default_value'     => 'Book a Strategy Call',
            ],
            [
                'key'               => 'field_primary_cta_url',
```

```
                'label'             => 'Primary CTA Button URL',
                'name'              => 'primaryCtaUrl',
                'type'              => 'text',
                'show_in_graphql'   => 1,
                'default_value'     => '/contact/',
            ],
        ],
        'location' => [
            [
                [
                    'param'    => 'post_type',
                    'operator' => '==',
                    'value'    => 'page',
                ],
            ],
        ],
    ] );
} );

/**
 * 4. TRIGGER VERCEL REBUILD WEBHOOK ON POST OR PAGE PUBLISH / UPDATE
 * Why: Since Astro builds static HTML, publishing or editing a WordPress page/post
 * needs to notify Vercel to run a fresh build automatically.
 * Note: Define VERCEL_DEPLOY_HOOK_URL in wp-config.php.
 */
add_action( 'save_post', function( $post_id, $post, $update ) {
    if ( wp_is_post_revision( $post_id ) || wp_is_post_autosave( $post_id ) ) {
        return;
    }
    if ( ! in_array( $post->post_type, [ 'post', 'page' ], true ) || $post->post_status !== 'publish' ) {
        return;
    }

    if ( defined( 'VERCEL_DEPLOY_HOOK_URL' ) && ! empty( VERCEL_DEPLOY_HOOK_URL ) ) {
        wp_remote_post( VERCEL_DEPLOY_HOOK_URL, [
            'timeout'  => 5,
            'blocking' => false,
            'headers'  => [ 'Content-Type' => 'application/json' ],
            'body'     => wp_json_encode( [
                'event'     => 'content_published',
                'post_type' => $post->post_type,
                'slug'      => $post->post_name,
            ] ),
        ] );
    }
}, 10, 3 );
```

Step 3: Create the starter pages (`Home` and `About`) in WordPress

1. In `/wp-admin`, go to **Pages → Add New Page**.
2. Create a page titled **Home** (ensure its permalink slug is `home`), fill in the **Template Page Content (Headless)** fields at the bottom, and click **Publish**.
3. Create a second page titled **About** (ensure its permalink slug is `about`), fill in its fields and body text, and click **Publish**.

4. Go to **Settings** → **Reading**. Under *Your homepage displays*, select **A static page**, choose **Home** as the Homepage, and click **Save Changes**.

2.5 Permalink, REST & GraphQL Endpoint Verification

WordPress GraphQL and REST routes **will return 404 errors** if WordPress is using "Plain" (`?p=123`) permalinks.

1. Set permalinks

1. In `/wp-admin`, navigate to **Settings** → **Permalinks**.
2. Select **Post name** (`https://wp.<domain.com>/sample-post/`) or **Custom Structure** (`/%postname%/`).
3. Click **Save Changes** (even if it is already selected, clicking Save flushes WordPress rewrite rules).

2. Verify the REST API endpoint

TERMINAL

```
curl -I https://wp.goodrep.media/wp-json/wp/v2/posts
```

Expected result: `HTTP/2 200` with `content-type: application/json; charset=UTF-8`.

3. Verify the WPGraphQL endpoint

In `/wp-admin`, click **GraphQL** → **Settings** and check **"Enable Public Introspection"** (optional for local dev, helpful for schema inspection). Then open **GraphQL** → **GraphiQL IDE**, or run this command from your terminal:

TERMINAL

```
curl -X POST https://wp.goodrep.media/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ generalSettings { title url } pages(first: 5) { nodes { title slug } } }"}'
```

Expected result: a `200 OK` JSON response containing `{"data":{"generalSettings":{"title":"..."},"pages":{"nodes":[...]}}}`.

2.6 Critical WordPress Settings Often Missed

1. Discourage search engines on `wp.<domain.com>` (prevents a duplicate-content SEO penalty)

Because `wp.<domain.com>` is only our headless backend and `https://<domain.com>` (on Vercel) is the public website, you **must** prevent Google from indexing `wp.<domain.com>`. Go to **Settings** → **Reading**, check **"Discourage search engines from indexing this site"**, and click **Save Changes**.

Yoast SEO may show a warning banner in `/wp-admin` when this is checked. That is expected on the headless backend subdomain, because Astro generates our production `<meta name="robots" content="index,follow">` tags directly on Vercel.

2. Wordfence / ModSecurity WAF whitelisting

As discovered during our Good Rep Media rollout, security plugins like **Wordfence** or server firewalls (**Imunify360 / ModSecurity**) often block unauthenticated `POST` requests to `/graphql` with `HTTP 403 Forbidden`. If using Wordfence, go to **Wordfence → All Options → Whitelisted URLs** and add `/graphql` for `POST` requests, or switch Wordfence to Learning Mode while testing form submissions.

3. Site Address (URL) vs. WordPress Address (URL)

In **Settings → General**, keep both **WordPress Address (URL)** and **Site Address (URL)** set to `https://wp.<domain.com>` so `/wp-admin` and media uploads resolve cleanly without redirect loops.

4. Editor user roles

Create accounts for content writers using the **Editor** role (not **Administrator**). Editors can publish pages, blog posts, and edit ACF fields, but cannot deactivate `WPGraphQL` or break server settings.

3 Frontend Project Setup

3.1 Project Creation Commands

Open your terminal, navigate to your workspace folder, and run these commands in order:

TERMINAL

```
# Step 1: Verify your Node.js version is >= 22.12.0
node -v

# Step 2: Scaffold a new minimal Astro project
npm create astro@latest marketing-site-template -- --template minimal --no-install --no-git

# Step 3: Enter the project directory
cd marketing-site-template

# Step 4: Initialize Git on the 'main' branch
git init -b main
```

3.2 Exact Dependency Installation & `package.json`

Run the following command to install the exact production-tested stack from our Good Rep Media reference project:

TERMINAL

```
# Install core runtime integrations, React 19 islands, Fontsource fonts, and security utilities
npm install astro@^7.3.8 @astrojs/react@^7.0.1 @astrojs/sitemap@^3.7.4 react@^19.3.1 react-dom@^19.3.1
@types/react@^19.3.0 @types/react-dom@^19.3.0 @fontsource/open-sans@^5.3.0 @fontsource/poppins@^5.3.0
gsap@^3.15.0 isomorphic-dompurify@^3.19.0 material-symbols@^0.40.2
```

Verify that your `package.json` matches this structure (including the `"engines"` constraint so Vercel automatically selects Node 22+):

JSON

```
{
  "name": "marketing-site-template",
  "type": "module",
  "version": "0.1.0",
  "engines": {
    "node": ">=22.12.0"
  },
  "scripts": {
    "dev": "astro dev",
    "build": "astro build",
    "preview": "astro preview",
    "astro": "astro"
  },
  "dependencies": {
    "@astrojs/react": "^7.0.1",
    "@astrojs/sitemap": "^3.7.4",
    "@fontsource/open-sans": "^5.3.0",
    "@fontsource/poppins": "^5.3.0",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "astro": "^7.3.8",
    "gsap": "^3.15.0",
    "isomorphic-dompurify": "^3.19.0",
    "material-symbols": "^0.40.2",
    "react": "^19.3.1",
    "react-dom": "^19.3.1"
  }
}
```

Next, update `astro.config.mjs` in the project root so Astro enables React islands, enforces trailing slashes (critical for SEO canonical consistency), and generates an XML sitemap:

astro.config.mjs JAVASCRIPT

```
// astro.config.mjs
import { defineConfig } from 'astro/config';
import react from '@astrojs/react';
import sitemap from '@astrojs/sitemap';

const SITE_URL = process.env.PUBLIC_SITE_URL || 'https://goodrep.media';

export default defineConfig({
  site: SITE_URL,
  trailingSlash: 'always',
  image: {
    // Allow Astro's <Image /> optimizer to process images hosted on WordPress
    domains: ['wp.goodrep.media', 'goodrep.media'],
  },
  integrations: [
    react(),
    sitemap({
      // Exclude private conversion routes from sitemap.xml
      filter: (page) => !page.includes('/thankyou/'),
    }),
```

```
  ],
});
```

3.3 Folder Structure Explanation

Create the following directory layout inside your project. Each directory has a single, clear responsibility:

PATH / STRUCTURE

```
marketing-site-template/
├── public/                    # Static assets copied as-is (favicons, subset fonts, robots.txt)
├── src/
│   ├── assets/                # Build-optimized images processed by Astro's <Image /> pipeline
│   ├── components/            # Reusable UI blocks (.astro static & .jsx React islands)
│   │   ├── BaseHead.astro     # Canonical URLs, OpenGraph, <ClientRouter />, JSON-LD schema
│   │   ├── TrackingHead.astro # Global GA4 (G-XXXXXXXXXX) script
│   │   ├── Header.astro       # Site navigation bar
│   │   ├── Footer.astro       # Site footer with copyright and legal links
│   │   └── ContactForm.jsx    # Interactive React 19 form submitting to WPGraphQL
│   ├── config/
│   │   └── site.js            # Central brand constants (name, phone, address, nav links)
│   ├── data/                  # Fallback/structured static copy so pages never crash offline
│   ├── layouts/
│   │   └── MainLayout.astro   # Master HTML shell: Header + <slot /> + Footer
│   ├── lib/
│   │   └── api.js             # Universal WPGraphQL + REST client with fallbacks
│   ├── pages/                 # File-based routing (each file becomes a URL route)
│   │   ├── index.astro        # Homepage      -> /
│   │   ├── about.astro        # About page    -> /about/
│   │   ├── contact.astro      # Contact page  -> /contact/
│   │   └── thankyou.astro     # Conversion page (noindex) -> /thankyou/
│   └── styles/
│       └── global.css         # CSS variables (tokens), reset, and utility classes
├── .env                       # Local environment variables (DO NOT commit secrets)
├── .gitignore                 # Excludes node_modules/, dist/, .astro/, and .env*
├── astro.config.mjs           # Astro configuration
├── package.json               # Dependencies and scripts
└── vercel.json                # Vercel edge trailing-slash and 301 redirect rules
```

Create the directories now with one command:

TERMINAL

```
mkdir -p src/assets src/components src/config src/data src/layouts src/lib src/pages src/styles
public/fonts
```

3.4 Connecting Astro to WPGraphQL (`src/lib/api.js`)

Why we don't need a heavy external GraphQL library (like Apollo Client): GraphQL queries are standard HTTP `POST` requests with a JSON body (`{ query, variables }`). Using Node's native `fetch()` keeps our bundle size at **0 KB** and lets us build a **universal dual-mode client** that speaks **WPGraphQL** first and automatically falls back to the **WordPress REST API** or **default starter data** if the CMS is unreachable during local setup.

Step 1: Create your `.env` file in the project root

.env — .ENV

```
# .env
PUBLIC_SITE_URL=https://goodrep.media
PUBLIC_WORDPRESS_API_URL=https://wp.goodrep.media/graphql
PUBLIC_WORDPRESS_GRAPHQL_URL=https://wp.goodrep.media/graphql
PUBLIC_GA_MEASUREMENT_ID=G-JP63DEKVVF
```

Step 2: Create `src/lib/api.js` and paste the complete, documented data client below:

src/lib/api.js — JAVASCRIPT

```
// src/lib/api.js
/**
 * Universal WordPress Data Client (WPGraphQL + REST Fallback + Safe Offline Defaults)
 *
 * Why this architecture works:
 * 1. Primary path: Queries WPGraphQL (/graphql) for structured Page, ACF, and Post data.
 * 2. Secondary path: If WPGraphQL is blocked or not yet active, falls back to WP REST (/wp-json/wp/v2).
 * 3. Offline fallback: If local WordPress isn't running yet, returns safe starter data so a junior
 *    developer can still run `npm run dev` and render Home, About, and Contact immediately.
 */

const RAW_API_URL = (
  (typeof import.meta !== 'undefined' && import.meta.env?.PUBLIC_WORDPRESS_API_URL) ||
  (typeof process !== 'undefined' && process.env?.PUBLIC_WORDPRESS_API_URL) ||
  'https://wp.goodrep.media/graphql'
).replace(/\/$/, '');

const isGraphQL = RAW_API_URL.endsWith('/graphql');
const REST_BASE_URL = isGraphQL
  ? `${RAW_API_URL.replace(/\/graphql$/, '')}/wp-json/wp/v2`
  : RAW_API_URL;

/**
 * Fallback starter data used if the WordPress CMS is unreachable or the page hasn't been created yet.
 */
export const FALLBACK_PAGES = {
  home: {
    title: 'Home',
    slug: 'home',
    content:
      '<p>We partner with ambitious businesses to build high-converting websites, dominate search
rankings, and scale profitable paid media campaigns.</p>',
    seo: {
```

```
      title: 'Good Rep Media | Digital Marketing & Web Design Agency',
      metaDesc:
        'Full-service digital marketing agency specializing in Astro headless web design, technical SEO,
and lead generation.',
    },
    templatePageFields: {
      heroKicker: 'FULL-SERVICE DIGITAL GROWTH PARTNER',
      heroHeadline: 'Marketing Engineered for Measurable Revenue',
      heroSubheadline:
        'Sub-second websites, transparent SEO, and conversion-focused campaigns tailored for growing
businesses.',
      primaryCtaLabel: 'Request a Free Strategy Call',
      primaryCtaUrl: '/contact/',
    },
  },
  about: {
    title: 'About Us',
    slug: 'about',
    content:
      '<p>We founded our agency on a simple premise: businesses deserve honest, high-caliber digital
marketing without corporate jargon, bloated retainers, or slow templates.</p><p>Every strategy we deploy
is tied directly to qualified leads and measurable return on investment.</p>',
    seo: {
      title: 'About Us | Our Story, Mission & Marketing Philosophy',
      metaDesc:
        'Learn how our team builds long-term partnerships and high-performing digital assets for small and
mid-sized businesses.',
    },
    templatePageFields: {
      heroKicker: 'ABOUT OUR AGENCY',
      heroHeadline: 'A Marketing Team That Feels Like Part of Yours',
      heroSubheadline:
        'Hands-on strategy, radical transparency, and senior execution on every account.',
      primaryCtaLabel: 'Work With Our Team',
      primaryCtaUrl: '/contact/',
    },
  },
};

/**
 * Decodes HTML entities (like &amp;#8217; or &amp;amp;) returned by WordPress fields.
 */
export function decodeHtmlEntities(str = '') {
  if (!str || typeof str !== 'string') return '';
  return str
    .replace(/&#(\d+);/g, (_, dec) => String.fromCharCode(Number(dec)))
    .replace(/&amp;/g, '&')
    .replace(/&quot;/g, '"')
    .replace(/&#039;|&apos;/g, "'")
    .replace(/&ndash;/g, '–')
    .replace(/&mdash;/g, '—')
    .replace(/&hellip;/g, '…')
    .replace(/ /g, ' ');
}

/**
 * Core GraphQL execution helper.
 * Sends an HTTP POST request with `{ query, variables }` to WPGraphQL.
 */
```

```
export async function fetchWP(query, variables = {}) {
  const endpoint = isGraphQL ? RAW_API_URL : `${RAW_API_URL.replace(/\/wp-json\/.*$/, '')}/graphql`;

  const response = await fetch(endpoint, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ query, variables }),
  });

  if (!response.ok) {
    throw new Error(`WPGraphQL HTTP Error: ${response.status} ${response.statusText}`);
  }

  const json = await response.json();
  if (json.errors) {
    console.error('[WPGraphQL Errors]:', JSON.stringify(json.errors, null, 2));
    throw new Error(json.errors[0]?.message || 'GraphQL query returned errors');
  }

  return json.data;
}

/**
 * Fetches a single WordPress Page by its slug (e.g., "home" or "about").
 * Priority order:
 * 1. WPGraphQL query (including Yoast SEO and ACF `templatePageFields`)
 * 2. WordPress REST API (`/wp-json/wp/v2/pages?slug=...`)
 * 3. Safe local fallback (`FALLBACK_PAGES[slug]`) so builds never crash when CMS is offline.
 */
export async function getPageBySlug(slug) {
  const cleanSlug = String(slug || '').toLowerCase().trim();
  const fallback = FALLBACK_PAGES[cleanSlug] || FALLBACK_PAGES.home;

  // Attempt 1: WPGraphQL
  if (isGraphQL) {
    try {
      const data = await fetchWP(
        `
        query GetPageBySlug($slug: ID!) {
          page(id: $slug, idType: URI) {
            title
            slug
            content
            seo {
              title
              metaDesc
            }
            templatePageFields {
              heroKicker
              heroHeadline
              heroSubheadline
              primaryCtaLabel
              primaryCtaUrl
            }
          }
        }
        `,
        { slug: cleanSlug }
      );
```

```
      if (data?.page) {
        return {
          title: decodeHtmlEntities(data.page.title || fallback.title),
          slug: data.page.slug || cleanSlug,
          content: data.page.content || fallback.content,
          seo: {
            title: decodeHtmlEntities(data.page.seo?.title || data.page.title || fallback.seo.title),
            metaDesc: decodeHtmlEntities(data.page.seo?.metaDesc || fallback.seo.metaDesc),
          },
          templatePageFields: {
            heroKicker: data.page.templatePageFields?.heroKicker ||
fallback.templatePageFields.heroKicker,
            heroHeadline: data.page.templatePageFields?.heroHeadline ||
fallback.templatePageFields.heroHeadline,
            heroSubheadline: data.page.templatePageFields?.heroSubheadline ||
fallback.templatePageFields.heroSubheadline,
            primaryCtaLabel: data.page.templatePageFields?.primaryCtaLabel ||
fallback.templatePageFields.primaryCtaLabel,
            primaryCtaUrl: data.page.templatePageFields?.primaryCtaUrl ||
fallback.templatePageFields.primaryCtaUrl,
          },
        };
      }
    } catch (graphqlError) {
      console.warn(`[CMS Notice] WPGraphQL lookup for "${cleanSlug}" unavailable
(${graphqlError.message}). Trying REST fallback...`);
    }
  }

  // Attempt 2: WordPress Native REST API Fallback
  try {
    const res = await fetch(`${REST_BASE_URL}/pages?slug=${encodeURIComponent(cleanSlug)}`);
    if (res.ok) {
      const pages = await res.json();
      if (Array.isArray(pages) && pages.length > 0) {
        const p = pages[0];
        return {
          title: decodeHtmlEntities(p.title?.rendered || fallback.title),
          slug: p.slug || cleanSlug,
          content: p.content?.rendered || fallback.content,
          seo: {
            title: decodeHtmlEntities(p.yoast_head_json?.title || p.title?.rendered ||
fallback.seo.title),
            metaDesc: decodeHtmlEntities(p.yoast_head_json?.description || fallback.seo.metaDesc),
          },
          templatePageFields: {
            heroKicker: p.acf?.heroKicker || fallback.templatePageFields.heroKicker,
            heroHeadline: p.acf?.heroHeadline || decodeHtmlEntities(p.title?.rendered) ||
fallback.templatePageFields.heroHeadline,
            heroSubheadline: p.acf?.heroSubheadline || fallback.templatePageFields.heroSubheadline,
            primaryCtaLabel: p.acf?.primaryCtaLabel || fallback.templatePageFields.primaryCtaLabel,
            primaryCtaUrl: p.acf?.primaryCtaUrl || fallback.templatePageFields.primaryCtaUrl,
          },
        };
      }
    }
  } catch (restError) {
    console.warn(`[CMS Notice] REST lookup for "${cleanSlug}" unavailable (${restError.message}). Using
```

```
safe local fallback.`);
  }

  // Attempt 3: Offline Fallback
  return fallback;
}

/**
 * Fetches latest blog posts for the Homepage preview section.
 */
export async function getRecentPosts(limit = 3) {
  if (isGraphQL) {
    try {
      const data = await fetchWP(
        `
        query GetRecentPosts($first: Int!) {
          posts(first: $first, where: { status: PUBLISH }) {
            nodes {
              databaseId
              title
              slug
              date
              excerpt
            }
          }
        }
        `,
        { first: limit }
      );
      if (data?.posts?.nodes?.length > 0) {
        return data.posts.nodes.map((post) => ({
          id: post.databaseId,
          title: decodeHtmlEntities(post.title),
          slug: post.slug,
          date: post.date,
          excerpt: post.excerpt || '',
        }));
      }
    } catch {
      // Fall through to REST
    }
  }

  try {
    const res = await fetch(
      `${REST_BASE_URL}/posts?_fields=id,slug,title,excerpt,date&per_page=${limit}&page=1`
    );
    if (res.ok) {
      const posts = await res.json();
      if (Array.isArray(posts) && posts.length > 0) {
        return posts.map((p) => ({
          id: p.id,
          title: decodeHtmlEntities(p.title?.rendered || ''),
          slug: p.slug,
          date: p.date,
          excerpt: p.excerpt?.rendered || '',
        }));
      }
    }
```

```
  } catch {
    // Return empty array if offline
  }

  return [];
}
```

4 Vercel Integration

4.1 Linking the Git Repository to Vercel

Step 1: Push your project to GitHub

TERMINAL

```
git add .
git commit -m "chore: initialize Astro headless WordPress starter template"
git remote add origin https://github.com/<your-org>/<your-repo>.git
git push -u origin main
```

Step 2: Import into Vercel

1. Log in to vercel.com and click **Add New…** → **Project**.
2. Under **Import Git Repository**, locate `<your-repo>` and click **Import**.

4.2 Build Settings, Output Directory & Edge Config (`vercel.json`)

On the Vercel **Configure Project** screen, verify the following settings (Vercel auto-detects Astro, but always double-check):

SETTING	EXPECTED VALUE	NOTES
Framework Preset	`Astro`	Automatically detected from `package.json`.
Root Directory	`./`	Leave as `./` unless inside a monorepo.
Build Command	`astro build` *(or `npm run build`)*	Compiles static HTML and optimizes images.
Output Directory	`dist`	Default Astro static build output folder.
Install Command	`npm install`	Installs dependencies from `package-lock.json`.
Node.js Version	`22.x`	Enforced via `"engines": { "node": ">=22.12.0" }` in `package.json`.

Create a `vercel.json` file in your project root to enforce trailing slashes (`https://domain.com/about/`) and configure edge 301 redirects:

```json
{
  "trailingSlash": true,
  "redirects": [
    {
      "source": "/home/",
      "destination": "/",
      "statusCode": 301
    }
  ]
}
```

4.3 Setting Environment Variables in Vercel

Before clicking **Deploy** (or under **Project Settings** → **Environment Variables**), add each variable below and check all three environment boxes (**Production**, **Preview**, and **Development**):

1. `PUBLIC_SITE_URL` = `https://<domain.com>`
2. `PUBLIC_WORDPRESS_API_URL` = `https://wp.<domain.com>/graphql`
3. `PUBLIC_WORDPRESS_GRAPHQL_URL` = `https://wp.<domain.com>/graphql`
4. `PUBLIC_GA_MEASUREMENT_ID` = `G-XXXXXXXXXX` *(use the client's specific GA4 stream Measurement ID, and never reuse another client's ID!)*

Click **Deploy**.

4.4 Connecting Custom / Staging Domains via WHM & cPanel DNS

When connecting a staging subdomain (e.g., `staging2.<domain.com>`) or production domain (`<domain.com>`) where DNS is hosted on WHM/cPanel:

> GOLDEN RULE
>
> **GOLDEN RULE (from the production SOP)**
> **NEVER** go to **cPanel** → **Domains** → **Create A New Domain** for a frontend domain hosted on Vercel.
> Doing so creates a local webroot on the cPanel server and points traffic to the cPanel IP instead of Vercel.
> **ALWAYS** go directly to **cPanel** → **Zone Editor** (or **WHM** → **DNS Zone Manager**).

1. **In Vercel:** go to **Project** → **Settings** → **Domains**, type `staging2.<domain.com>` (or `<domain.com>`), and click **Add**.
2. **Copy the exact project-specific DNS target.** For a subdomain (`staging2`), Vercel provides a `CNAME` record (e.g., `<unique-hash>.vercel-dns-017.com.`). For an apex domain (`@`), Vercel provides an `A` record (e.g., `76.76.21.21`).

3. **In cPanel → Zone Editor:** click **Manage** next to `<domain.com>`, click **+ Add Record**, select **CNAME** (staging subdomain) or **A** (apex production), paste the exact Name and Value from Vercel, and click **Save Record**.
4. **Configure the WordPress rebuild webhook (Deploy Hook):** in Vercel, go to **Settings → Git → Deploy Hooks**. Enter Name `WordPress Content Publish` and Branch `main`, then click **Create Hook**. Copy the generated URL and add it to your WordPress `wp-config.php`:

```php
define( 'VERCEL_DEPLOY_HOOK_URL', 'https://api.vercel.com/v1/integrations/deploy/prj_xxx/yyy' );
```

4.5 Verifying Preview & Production Deployments

Step 1: Create a test feature branch locally

```terminal
git checkout -b test/preview-check
git commit --allow-empty -m "test: trigger Vercel preview build"
git push -u origin test/preview-check
```

Step 2: Open your Vercel Dashboard and go to **Deployments**.

Step 3: Confirm that a **Preview** deployment automatically starts, completes with `✓ Build Completed`, and generates a unique `https://<project>-git-test-preview-check-<team>.vercel.app` URL.

5 Basic Template Layout (Copy-Pasteable Starter Code)

Below is the complete, copy-pasteable code for the minimal working base. Every file includes inline comments explaining how and why it works.

5.1 Site Configuration (`src/config/site.js`)

Create `src/config/site.js` to store brand metadata and navigation links in one central place:

src/config/site.js JAVASCRIPT

```
// src/config/site.js
/**
 * Central Brand & Navigation Configuration
 * When cloning this template for a new client, update these values first.
 */

export const SITE_NAME = 'Good Rep Media';
export const SITE_URL = import.meta.env.PUBLIC_SITE_URL || 'https://goodrep.media';

export const DEFAULT_PAGE_TITLE = `${SITE_NAME} | Digital Marketing & Web Design Agency`;
export const DEFAULT_META_DESCRIPTION =
  'High-converting decoupled websites, technical SEO, and paid media management engineered for measurable
business growth.';

export const CONTACT_INFO = Object.freeze({
  phoneDisplay: '(616) 300-9868',
  phoneHref: 'tel:+16163009868',
  email: 'info@goodrep.media',
  streetAddress: '0-745 Lake Michigan Dr. NW',
  city: 'Grand Rapids',
  state: 'MI',
  postalCode: '49534',
  country: 'US',
});

export const NAV_LINKS = Object.freeze([
  { label: 'Home', href: '/' },
  { label: 'About', href: '/about/' },
  { label: 'Contact', href: '/contact/' },
]);
```

5.2 Head & Analytics Components (`BaseHead.astro` & `TrackingHead.astro`)

1. Create `src/components/BaseHead.astro`. This component manages SEO metadata, canonical URLs with enforced trailing slashes, Astro's `<ClientRouter />` (which fires `astro:page-load` events on navigation), and JSON-LD structured data:

src/components/BaseHead.astro ASTRO

```astro
---
// src/components/BaseHead.astro
import { DEFAULT_PAGE_TITLE, DEFAULT_META_DESCRIPTION, SITE_URL } from '../config/site.js';
import { ClientRouter } from 'astro:transitions';

const {
  title = DEFAULT_PAGE_TITLE,
  description = DEFAULT_META_DESCRIPTION,
  image = '/favicon.svg',
  noindex = false,
  schemaGraph = null,
} = Astro.props;

// Ensure canonical URL always ends with a trailing slash to match vercel.json & astro.config.mjs
let pathname = Astro.url.pathname;
if (!pathname.endsWith('/')) {
  pathname += '/';
}
const canonical = new URL(pathname, SITE_URL).href;
const ogImage = new URL(image, SITE_URL).href;

// Automatically noindex staging builds if GITHUB_PAGES is set
const isStaging = import.meta.env.GITHUB_PAGES === 'true';
const shouldNoIndex = noindex || isStaging;
---

<meta charset="utf-8" />
<meta name="viewport" content="width=device-width,initial-scale=1" />
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
<meta name="theme-color" content="#0b1528" />

<title>{title}</title>
<meta name="description" content={description} />
<link rel="canonical" href={canonical} />

<!-- Enables smooth SPA view transitions and fires 'astro:page-load' lifecycle events -->
<ClientRouter />

<!-- Prevent search engines from indexing thank-you pages or staging environments -->
{shouldNoIndex && <meta name="robots" content="noindex,follow" />}

<!-- Open Graph / Social Sharing Metadata -->
<meta property="og:type" content="website" />
<meta property="og:title" content={title} />
<meta property="og:description" content={description} />
<meta property="og:url" content={canonical} />
<meta property="og:image" content={ogImage} />

<!-- Twitter Card Metadata -->
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:title" content={title} />
<meta name="twitter:description" content={description} />
<meta name="twitter:image" content={ogImage} />

<!-- Composable JSON-LD Structured Data Graph -->
{schemaGraph && (
```

```
  <script type="application/ld+json" set:html={JSON.stringify(schemaGraph)} />
)}
```

2. Create `src/components/TrackingHead.astro`. This component loads the global Google Analytics 4 (`gtag.js`) base tag using the environment variable `PUBLIC_GA_MEASUREMENT_ID`. Lead conversion events are **never** placed here; they fire exclusively on `/thankyou/`.

src/components/TrackingHead.astro — ASTRO

```
---
// src/components/TrackingHead.astro
// Global Analytics Base Script.
// IMPORTANT: Always set PUBLIC_GA_MEASUREMENT_ID in .env and Vercel so client properties never cross-
pollinate.
const GA_MEASUREMENT_ID = import.meta.env.PUBLIC_GA_MEASUREMENT_ID || 'G-JP63DEKVVF';
---

<!-- Google tag (gtag.js) - Google Analytics 4 -->
<script async src={`https://www.googletagmanager.com/gtag/js?id=${GA_MEASUREMENT_ID}`}></script>
<script is:inline define:vars={{ GA_MEASUREMENT_ID }}>
  window.dataLayer = window.dataLayer || [];
  function gtag(){dataLayer.push(arguments);}
  gtag('js', new Date());
  gtag('config', GA_MEASUREMENT_ID);
</script>
```

5.3 Starter Components (`Header.astro`, `Footer.astro`, `ContactForm.jsx`)

1. Create `src/components/Header.astro`

src/components/Header.astro — ASTRO

```
---
// src/components/Header.astro
import { SITE_NAME, NAV_LINKS, CONTACT_INFO } from '../config/site.js';

// Highlight the current active page in the navigation bar
const currentPath = Astro.url.pathname;
---

<header class="site-header">
  <div class="container site-header__inner">
    <!-- Brand Logo Link -->
    <a href="/" class="site-header__brand" aria-label={`${SITE_NAME} Home`}>
      <span class="site-header__brand-name">{SITE_NAME}</span>
    </a>

    <!-- Primary Navigation -->
    <nav class="site-nav" aria-label="Primary Navigation">
      <ul class="site-nav__list">
        {NAV_LINKS.map((item) => {
          const isActive = currentPath === item.href || (item.href !== '/' &&
currentPath.startsWith(item.href));
```

```
          return (
            <li>
              <a
                href={item.href}
                class={`site-nav__link ${isActive ? 'site-nav__link--active' : ''}`}
                aria-current={isActive ? 'page' : undefined}
              >
                {item.label}
              </a>
            </li>
          );
        })}
      </ul>
    </nav>

    <!-- Header Direct Call & Primary CTA -->
    <div class="site-header__actions">
      <a href={CONTACT_INFO.phoneHref} class="site-header__phone">
        {CONTACT_INFO.phoneDisplay}
      </a>
      <a href="/contact/" class="btn btn--primary">Get a Quote</a>
    </div>
  </div>
</header>
```

2. Create `src/components/Footer.astro`

src/components/Footer.astro — ASTRO

```
---
// src/components/Footer.astro
import { SITE_NAME, NAV_LINKS, CONTACT_INFO } from '../config/site.js';
const currentYear = new Date().getFullYear();
---

<footer class="site-footer">
  <div class="container site-footer__grid">
    <!-- Column 1: Brand Info -->
    <div class="site-footer__col">
      <strong class="site-footer__title">{SITE_NAME}</strong>
      <p class="site-footer__desc">
        Decoupled Headless WordPress &amp;amp; Astro marketing website template engineered for speed, SEO, and
conversions.
      </p>
    </div>

    <!-- Column 2: Quick Links -->
    <div class="site-footer__col">
      <h2 class="site-footer__heading">Navigation</h2>
      <ul class="site-footer__links">
        {NAV_LINKS.map((item) => (
          <li><a href={item.href}>{item.label}</a></li>
        ))}
      </ul>
    </div>

    <!-- Column 3: Contact Details -->
```

```
    <div class="site-footer__col">
      <h2 class="site-footer__heading">Contact Us</h2>
      <address class="site-footer__address">
        <p>{CONTACT_INFO.streetAddress}</p>
        <p>{CONTACT_INFO.city}, {CONTACT_INFO.state} {CONTACT_INFO.postalCode}</p>
        <p><a href={CONTACT_INFO.phoneHref}>{CONTACT_INFO.phoneDisplay}</a></p>
        <p><a href={`mailto:${CONTACT_INFO.email}`}>{CONTACT_INFO.email}</a></p>
      </address>
    </div>
  </div>

  <div class="container site-footer__bottom">
    <p>&copy; {currentYear} {SITE_NAME}. All rights reserved.</p>
  </div>
</footer>
```

3. Create `src/components/ContactForm.jsx` (interactive React 19 island). This component provides a working contact form placeholder that validates user input, formats US phone numbers automatically (`(XXX) XXX-XXXX`), submits to WordPress via the `submitGfForm` WPGraphQL mutation (matching our production `LeadForm.jsx` architecture), and redirects to `/thankyou/` upon completion:

src/components/ContactForm.jsx — REACT JSX

```jsx
// src/components/ContactForm.jsx
import { useState, useId } from 'react';
import DOMPurify from 'isomorphic-dompurify';

/**
 * Standard Gravity Forms Field IDs matching our WordPress backend form configuration.
 */
export const GF_FIELD_IDS = Object.freeze({
  COMPANY: 4,
  EMAIL: 5,
  PHONE: 6,
  MESSAGE: 9,
  FIRST_NAME: 12,
  LAST_NAME: 13,
});

const SUBMIT_GF_FORM_MUTATION = `
  mutation SubmitLeadForm($input: SubmitGfFormInput!) {
    submitGfForm(input: $input) {
      confirmation {
        type
        message
        url
      }
      errors {
        id
        message
      }
    }
  }
`;

/**
```

```
 * Resolves the WPGraphQL endpoint from environment variables.
 */
function getGraphQLEndpoint() {
  if (typeof import.meta !== 'undefined') {
    if (import.meta.env?.PUBLIC_WORDPRESS_GRAPHQL_URL) {
      return import.meta.env.PUBLIC_WORDPRESS_GRAPHQL_URL;
    }
    if (import.meta.env?.PUBLIC_WORDPRESS_API_URL) {
      const url = import.meta.env.PUBLIC_WORDPRESS_API_URL;
      if (url.endsWith('/graphql')) return url;
      return `${url.replace(/\/wp-json\/.*$/, '')}/graphql`;
    }
  }
  return 'https://wp.goodrep.media/graphql';
}

/**
 * Formats raw digits into US phone format: (XXX) XXX-XXXX
 */
export function formatPhoneNumber(value = '') {
  let digits = value.replace(/\D/g, '');
  if (digits.length === 11 && digits.startsWith('1')) {
    digits = digits.slice(1);
  }
  digits = digits.slice(0, 10);
  if (digits.length === 0) return '';
  if (digits.length <= 3) return `(${digits}`;
  if (digits.length <= 6) return `(${digits.slice(0, 3)}) ${digits.slice(3)}`;
  return `(${digits.slice(0, 3)}) ${digits.slice(3, 6)}-${digits.slice(6, 10)}`;
}

export default function ContactForm({ formId = 15, submitLabel = 'Send Message' }) {
  const uid = useId();
  const [formData, setFormData] = useState({
    firstName: '',
    lastName: '',
    email: '',
    phone: '',
    company: '',
    message: '',
  });
  const [status, setStatus] = useState('idle'); // 'idle' | 'submitting' | 'success' | 'error'
  const [errorMessage, setErrorMessage] = useState('');

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: name === 'phone' ? formatPhoneNumber(value) : value,
    }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setStatus('submitting');
    setErrorMessage('');

    try {
      const response = await fetch(getGraphQLEndpoint(), {
```

```
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          query: SUBMIT_GF_FORM_MUTATION,
          variables: {
            input: {
              id: String(formId),
              fieldValues: [
                { id: GF_FIELD_IDS.FIRST_NAME, value: formData.firstName.trim() },
                { id: GF_FIELD_IDS.LAST_NAME, value: formData.lastName.trim() },
                { id: GF_FIELD_IDS.EMAIL, emailValues: { value: formData.email.trim() } },
                { id: GF_FIELD_IDS.PHONE, value: formData.phone.trim() },
                { id: GF_FIELD_IDS.COMPANY, value: formData.company.trim() },
                { id: GF_FIELD_IDS.MESSAGE, value: DOMPurify.sanitize(formData.message.trim()) },
              ],
            },
          },
        }),
      });

      const result = await response.json();
      const gfErrors = result?.data?.submitGfForm?.errors;

      if (gfErrors && gfErrors.length > 0) {
        throw new Error(gfErrors[0].message || 'Validation failed on the server.');
      }

      if (result.errors) {
        throw new Error(result.errors[0]?.message || 'Could not submit form.');
      }

      setStatus('success');
      // Redirect to isolated conversion page so GA4 fires 'generate_lead'
      window.location.assign('/thankyou/');
    } catch (err) {
      setStatus('error');
      setErrorMessage(
        err.message || 'Unable to reach the form server. Please call us directly.'
      );
    }
  };

  return (
    <form className="contact-form" onSubmit={handleSubmit} noValidate>
      <div className="contact-form__row">
        <div className="contact-form__field">
          <label htmlFor={`${uid}-firstName`}>First Name *</label>
          <input
            id={`${uid}-firstName`}
            name="firstName"
            type="text"
            required
            value={formData.firstName}
            onChange={handleChange}
            placeholder="Jane"
          />
        </div>
        <div className="contact-form__field">
          <label htmlFor={`${uid}-lastName`}>Last Name *</label>
```

```
            <input
              id={`${uid}-lastName`}
              name="lastName"
              type="text"
              required
              value={formData.lastName}
              onChange={handleChange}
              placeholder="Doe"
            />
          </div>
        </div>

        <div className="contact-form__row">
          <div className="contact-form__field">
            <label htmlFor={`${uid}-email`}>Work Email *</label>
            <input
              id={`${uid}-email`}
              name="email"
              type="email"
              required
              value={formData.email}
              onChange={handleChange}
              placeholder="jane@company.com"
            />
          </div>
          <div className="contact-form__field">
            <label htmlFor={`${uid}-phone`}>Phone Number *</label>
            <input
              id={`${uid}-phone`}
              name="phone"
              type="tel"
              required
              value={formData.phone}
              onChange={handleChange}
              placeholder="(555) 234-5678"
            />
          </div>
        </div>

        <div className="contact-form__field">
          <label htmlFor={`${uid}-company`}>Company / Organization</label>
          <input
            id={`${uid}-company`}
            name="company"
            type="text"
            value={formData.company}
            onChange={handleChange}
            placeholder="Acme Corp"
          />
        </div>

        <div className="contact-form__field">
          <label htmlFor={`${uid}-message`}>How can we help you grow?</label>
          <textarea
            id={`${uid}-message`}
            name="message"
            rows={4}
            value={formData.message}
            onChange={handleChange}
```

```
          placeholder="Tell us about your project goals, timeline, or questions..."
        />
      </div>

      {status === 'error' && (
        <p className="contact-form__error" role="alert">
          {errorMessage}
        </p>
      )}

      <button
        type="submit"
        className="btn btn--primary contact-form__submit"
        disabled={status === 'submitting'}
      >
        {status === 'submitting' ? 'Sending...' : submitLabel}
      </button>
    </form>
  );
}
```

5.4 Master Layout Wrapper (`src/layouts/MainLayout.astro`)

1. Create `src/styles/global.css`. A clean, minimal stylesheet that sets up CSS custom properties, our 1600px container standard, and responsive layout primitives:

src/styles/global.css — CSS

```css
/* src/styles/global.css */
:root {
  --color-primary: #0b1528;
  --color-cyan: #1bc4d9;
  --color-bg: #f8fafc;
  --color-surface: #ffffff;
  --color-text: #1e293b;
  --color-muted: #64748b;
  --color-border: #e2e8f0;
  --font-heading: 'Poppins', system-ui, -apple-system, sans-serif;
  --font-body: 'Open Sans', system-ui, -apple-system, sans-serif;
  --container: 1600px;
}

*,
*::before,
*::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html,
body {
  font-family: var(--font-body);
  color: var(--color-text);
  background-color: var(--color-bg);
```

```
  line-height: 1.6;
  overflow-x: clip;
}

h1, h2, h3, h4 {
  font-family: var(--font-heading);
  color: var(--color-primary);
  line-height: 1.2;
}

.container {
  width: 100%;
  max-width: var(--container);
  margin-inline: auto;
  padding-inline: 1.5rem;
}

/* Header & Navigation */
.site-header {
  background: var(--color-primary);
  color: #ffffff;
  padding-block: 1rem;
  position: sticky;
  top: 0;
  z-index: 100;
}

.site-header__inner {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1.5rem;
}

.site-header__brand {
  color: #ffffff;
  text-decoration: none;
  font-family: var(--font-heading);
  font-weight: 800;
  font-size: 1.25rem;
  letter-spacing: 0.04em;
}

.site-nav__list {
  display: flex;
  list-style: none;
  gap: 2rem;
}

.site-nav__link {
  color: #cbd5e1;
  text-decoration: none;
  font-weight: 600;
  transition: color 0.2s ease;
}

.site-nav__link:hover,
.site-nav__link--active {
  color: var(--color-cyan);
```

```
}

.site-header__actions {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.site-header__phone {
  color: #ffffff;
  text-decoration: none;
  font-weight: 600;
}

/* Buttons */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 0.75rem 1.5rem;
  border-radius: 9999px;
  font-family: var(--font-heading);
  font-weight: 600;
  text-decoration: none;
  border: 2px solid transparent;
  cursor: pointer;
  transition: transform 0.15s ease, opacity 0.15s ease;
}

.btn--primary {
  background: var(--color-cyan);
  color: var(--color-primary);
}

.btn--outline {
  border-color: var(--color-primary);
  color: var(--color-primary);
  background: transparent;
}

/* Page Sections */
.page-hero {
  padding-block: 5rem;
  background: var(--color-surface);
  border-bottom: 1px solid var(--color-border);
}

.page-hero__kicker {
  display: inline-block;
  font-size: 0.85rem;
  font-weight: 700;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color: var(--color-cyan);
  margin-bottom: 0.75rem;
}

.page-hero__title {
  font-size: clamp(2.25rem, 4vw, 3.5rem);
```

```
  margin-bottom: 1rem;
}

.page-hero__lead {
  font-size: 1.15rem;
  color: var(--color-muted);
  max-width: 65ch;
  margin-bottom: 2rem;
}

.page-section {
  padding-block: 4rem;
}

/* Contact Form */
.contact-form {
  background: var(--color-surface);
  padding: 2rem;
  border-radius: 1rem;
  border: 1px solid var(--color-border);
  display: grid;
  gap: 1.25rem;
  max-width: 680px;
}

.contact-form__row {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

.contact-form__field {
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
}

.contact-form__field label {
  font-weight: 600;
  font-size: 0.9rem;
}

.contact-form__field input,
.contact-form__field textarea {
  padding: 0.75rem;
  border: 1px solid var(--color-border);
  border-radius: 0.5rem;
  font: inherit;
}

.contact-form__error {
  color: #dc2626;
  font-weight: 600;
}

/* Footer */
.site-footer {
  background: var(--color-primary);
  color: #cbd5e1;
```

```
  padding-block: 4rem 2rem;
  margin-top: 4rem;
}

.site-footer__grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 2.5rem;
  margin-bottom: 3rem;
}

.site-footer__title,
.site-footer__heading {
  color: #ffffff;
  font-size: 1.1rem;
  margin-bottom: 0.75rem;
  display: block;
}

.site-footer__links {
  list-style: none;
  display: grid;
  gap: 0.5rem;
}

.site-footer__links a,
.site-footer__address a {
  color: #cbd5e1;
  text-decoration: none;
}

.site-footer__bottom {
  border-top: 1px solid rgba(255, 255, 255, 0.12);
  padding-top: 1.5rem;
  font-size: 0.875rem;
}
```

2. Create `src/layouts/MainLayout.astro`. This wrapper imports our self-hosted `@fontsource` fonts, builds the global `Organization` and `LocalBusiness` JSON-LD `@graph`, and wraps every page with `<Header />`, `<main><slot /></main>`, and `<Footer />`:

src/layouts/MainLayout.astro — ASTRO

```
---
// src/layouts/MainLayout.astro
import '@fontsource/open-sans/400.css';
import '@fontsource/open-sans/600.css';
import '@fontsource/open-sans/700.css';
import '@fontsource/poppins/600.css';
import '@fontsource/poppins/700.css';
import '@fontsource/poppins/800.css';
import '../styles/global.css';

import BaseHead from '../components/BaseHead.astro';
import TrackingHead from '../components/TrackingHead.astro';
import Header from '../components/Header.astro';
import Footer from '../components/Footer.astro';
```

```
import { SITE_NAME, SITE_URL, DEFAULT_META_DESCRIPTION, CONTACT_INFO } from '../config/site.js';

const {
  title = `${SITE_NAME} | Digital Marketing Agency`,
  description = DEFAULT_META_DESCRIPTION,
  image,
  noindex = false,
  pageSchema = null,
} = Astro.props;

// Global Organization & LocalBusiness Schema injected on every page
const globalSchema = {
  '@context': 'https://schema.org',
  '@graph': [
    {
      '@type': 'Organization',
      '@id': `${SITE_URL}/#organization`,
      name: SITE_NAME,
      url: `${SITE_URL}/`,
    },
    {
      '@type': 'LocalBusiness',
      '@id': `${SITE_URL}/#localbusiness`,
      name: SITE_NAME,
      url: `${SITE_URL}/`,
      telephone: CONTACT_INFO.phoneDisplay,
      address: {
        '@type': 'PostalAddress',
        streetAddress: CONTACT_INFO.streetAddress,
        addressLocality: CONTACT_INFO.city,
        addressRegion: CONTACT_INFO.state,
        postalCode: CONTACT_INFO.postalCode,
        addressCountry: CONTACT_INFO.country,
      },
    },
  ],
};

if (pageSchema) {
  if (Array.isArray(pageSchema)) {
    globalSchema['@graph'].push(...pageSchema);
  } else {
    globalSchema['@graph'].push(pageSchema);
  }
}
---

<!doctype html>
<html lang="en">
  <head>
    <BaseHead
      title={title}
      description={description}
      image={image}
      noindex={noindex}
      schemaGraph={globalSchema}
    />
    <TrackingHead />
  </head>
```

```
  <body>
    <Header />
    <main id="main-content">
      <slot />
    </main>
    <Footer />
  </body>
</html>
```

5.5 Core Pages (`index.astro` , `about.astro` , `contact.astro` , `thankyou.astro`)

1. Homepage (`src/pages/index.astro`). Fetches the `home` page fields and recent blog posts from WordPress via `src/lib/api.js` (with automatic fallback if the CMS is offline) and sanitizes HTML with `isomorphic-dompurify` :

src/pages/index.astro ASTRO

```
---
// src/pages/index.astro
import DOMPurify from 'isomorphic-dompurify';
import MainLayout from '../layouts/MainLayout.astro';
import ContactForm from '../components/ContactForm.jsx';
import { getPageBySlug, getRecentPosts } from '../lib/api.js';

// Fetch Home page content and latest 3 blog posts at build time
const homePage = await getPageBySlug('home');
const recentPosts = await getRecentPosts(3);

const fields = homePage.templatePageFields;
const cleanContent = DOMPurify.sanitize(homePage.content);
---

<MainLayout title={homePage.seo.title} description={homePage.seo.metaDesc}>
  <!-- Hero Section populated from WordPress ACF / GraphQL -->
  <section class="page-hero">
    <div class="container">
      <span class="page-hero__kicker">{fields.heroKicker}</span>
      <h1 class="page-hero__title">{fields.heroHeadline}</h1>
      <p class="page-hero__lead">{fields.heroSubheadline}</p>
      <a href={fields.primaryCtaUrl} class="btn btn--primary">
        {fields.primaryCtaLabel}
      </a>
    </div>
  </section>

  <!-- Main WordPress Gutenberg Content Block -->
  <section class="page-section">
    <div class="container">
      <h2>Why Partner With Us</h2>
      <div class="cms-content" set:html={cleanContent} />
    </div>
  </section>

  <!-- Latest Insights from WordPress Posts -->
  {recentPosts.length > 0 && (
```

```
    <section class="page-section">
      <div class="container">
        <h2>Latest Insights from WordPress</h2>
        <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap:
1.5rem; margin-top: 1.5rem;">
          {recentPosts.map((post) => (
            <article style="background: #fff; padding: 1.5rem; border-radius: 0.75rem; border: 1px solid
var(--color-border);">
              <h3 style="margin-bottom: 0.5rem;">{post.title}</h3>
              <div set:html={DOMPurify.sanitize(post.excerpt)} />
            </article>
          ))}
        </div>
      </div>
    </section>
  )}

  <!-- Embedded Consultation Form Island -->
  <section class="page-section">
    <div class="container">
      <h2 style="margin-bottom: 1rem;">Start a Conversation</h2>
      <ContactForm client:visible submitLabel="Request My Strategy Call" />
    </div>
  </section>
</MainLayout>
```

2. About page (`src/pages/about.astro`). Fetches the `about` page from WordPress via `getPageBySlug('about')`, falling back cleanly if the CMS is offline:

src/pages/about.astro — ASTRO

```
---
// src/pages/about.astro
import DOMPurify from 'isomorphic-dompurify';
import MainLayout from '../layouts/MainLayout.astro';
import { getPageBySlug } from '../lib/api.js';

const aboutPage = await getPageBySlug('about');
const fields = aboutPage.templatePageFields;
const cleanContent = DOMPurify.sanitize(aboutPage.content);
---

<MainLayout title={aboutPage.seo.title} description={aboutPage.seo.metaDesc}>
  <section class="page-hero">
    <div class="container">
      <span class="page-hero__kicker">{fields.heroKicker}</span>
      <h1 class="page-hero__title">{fields.heroHeadline}</h1>
      <p class="page-hero__lead">{fields.heroSubheadline}</p>
      <a href={fields.primaryCtaUrl} class="btn btn--primary">
        {fields.primaryCtaLabel}
      </a>
    </div>
  </section>

  <section class="page-section">
    <div class="container">
      <h2 style="margin-bottom: 1rem;">Our Story &amp; Values</h2>
```

```
      <div class="cms-content" set:html={cleanContent} />
    </div>
  </section>
</MainLayout>
```

3. Contact page (`src/pages/contact.astro`). Renders the contact details and hydrates the interactive `ContactForm.jsx` React island on load (`client:load`):

src/pages/contact.astro — ASTRO

```
---
// src/pages/contact.astro
import MainLayout from '../layouts/MainLayout.astro';
import ContactForm from '../components/ContactForm.jsx';
import { SITE_NAME, CONTACT_INFO } from '../config/site.js';
---

<MainLayout
  title={`Contact Us | ${SITE_NAME}`}
  description={`Get in touch with ${SITE_NAME} to discuss your website, SEO, or paid media goals.`}
>
  <section class="page-hero">
    <div class="container">
      <span class="page-hero__kicker">GET IN TOUCH</span>
      <h1 class="page-hero__title">Let&rsquo;s Build Your Growth Roadmap</h1>
      <p class="page-hero__lead">
        Fill out the form below or call us directly at <a href={CONTACT_INFO.phoneHref}>
{CONTACT_INFO.phoneDisplay}</a>.
      </p>
    </div>
  </section>

  <section class="page-section">
    <div class="container">
      <ContactForm client:load submitLabel="Send Message" />
    </div>
  </section>
</MainLayout>
```

4. Isolated conversion thank-you page (`src/pages/thankyou.astro`). Matches our production Good Rep Media attribution funnel: sets `noindex={true}` so search engines never index it, and fires the standard GA4 `generate_lead` event only when a visitor lands here after submitting the form:

src/pages/thankyou.astro — ASTRO

```
---
// src/pages/thankyou.astro
import MainLayout from '../layouts/MainLayout.astro';
import { SITE_NAME } from '../config/site.js';
---

<MainLayout title={`Thank You | ${SITE_NAME}`} noindex={true}>
  <!-- Isolated GA4 Lead Conversion Event -->
  <script is:inline>
```

```
    window.addEventListener('load', function() {
      if (typeof gtag === 'function') {
        gtag('event', 'generate_lead', {
          event_category: 'engagement',
          event_label: 'Contact Form Submission'
        });
      }
    });
  </script>

  <section class="page-section" style="text-align: center; padding-block: 6rem;">
    <div class="container">
      <h1 style="margin-bottom: 1rem;">Message Received</h1>
      <p style="margin-bottom: 2rem; color: var(--color-muted);">
        Thank you for reaching out. A strategist will be in touch with you shortly.
      </p>
      <a href="/" class="btn btn--primary">&larr; Return to Homepage</a>
    </div>
  </section>
</MainLayout>
```

6 Verification Checklist

Tick off each item in order before marking Stage 1 complete:

- [ ] **1. Local environment verified:** `node -v` outputs `v22.12.0` or higher, and `npm install` finishes with 0 dependency errors.
- [ ] **2. WordPress backend running:** `https://wp.<domain.com>/wp-admin` (or LocalWP) is accessible, Permalinks are set to **Post name**, and "Discourage search engines from indexing this site" is checked in **Settings → Reading**.
- [ ] **3. Plugins & MU-plugin active:** `WPGraphQL`, `ACF`, `WPGraphQL for ACF`, `Yoast SEO`, and `wp-content/mu-plugins/headless-cms-config.php` are active.
- [ ] **4. GraphQL endpoint returns data:** running `curl -X POST https://wp.<domain.com>/graphql -H "Content-Type: application/json" -d '{"query":"{ pages { nodes { title slug } } }"}'` returns HTTP 200 with JSON nodes.
- [ ] **5. Local dev server runs (`npm run dev`):** opening `http://localhost:4321/` renders the Home page (`/`), About page (`/about/`), Contact page (`/contact/`), and Thank You page (`/thankyou/`).
- [ ] **6. Offline fallback tested:** temporarily changing `PUBLIC_WORDPRESS_API_URL` in `.env` to `https://invalid.local/graphql` still renders Home and About using `FALLBACK_PAGES` without crashing.
- [ ] **7. Production build succeeds (`npm run build`):** generates static HTML inside `dist/` and creates `dist/sitemap-index.xml` with 0 errors.
- [ ] **8. Vercel environment variables confirmed:** `PUBLIC_SITE_URL`, `PUBLIC_WORDPRESS_API_URL`, `PUBLIC_WORDPRESS_GRAPHQL_URL`, and `PUBLIC_GA_MEASUREMENT_ID` are set for Production, Preview, and Development in Vercel.
- [ ] **9. Vercel preview deployment succeeds:** pushing a branch to GitHub triggers an automated Vercel Preview build that resolves over HTTPS and submits the Contact Form without CORS errors.

7 Troubleshooting Guide

#	ERROR / SYMPTOM	ROOT CAUSE	EXACT FIX
1	**CORS error in browser console:** `Access to fetch at 'https://wp.../graphql' from origin 'https://...' has been blocked by CORS policy`	Origin domain or `*.vercel.app` preview URL is missing from `Access-Control-Allow-Origin` headers, or `OPTIONS` preflight requests are blocked.	Verify `wp-content/mu-plugins/headless-cms-config.php` is uploaded and contains your frontend domain in `$allowed_origins` plus the `OPTIONS` status 200 exit handler.
2	**HTTP 404 Not Found on `/graphql` or `/wp-json/wp/v2/posts`**	WordPress Permalinks are set to "Plain" (`?p=123`) or `.htaccess` rewrite rules were not flushed.	In WordPress `/wp-admin` , go to **Settings → Permalinks**, select **Post name**, and click **Save Changes**.
3	**HTTP 403 Forbidden when submitting `ContactForm.jsx` to `/graphql`**	Wordfence WAF, Imunify360, or ModSecurity is blocking anonymous `POST` payloads to `/graphql` .	In **Wordfence → All Options → Whitelisted URLs**, whitelist `/graphql` for `POST` requests, or ask hosting support to whitelist ModSecurity rule hits on `/graphql` .
4	**GraphQL error:** `Cannot query field "templatePageFields" on type "Page"`	`ACF` or `WPGraphQL for ACF` plugin is inactive, or `headless-cms-config.php` has not been placed in `wp-content/mu-plugins/` .	Activate both `Advanced Custom Fields` and `WPGraphQL for ACF` in `/wp-admin/plugins.php` and verify `headless-cms-config.php` is inside `wp-content/mu-plugins/` .
5	**Vercel build fails:** `Error: Unsupported Node.js version` or Astro module syntax error	Vercel project defaulted to an older Node.js runtime (e.g., Node 18) instead of Node 22+.	Ensure `"engines": { "node": ">=22.12.0" }` is present in `package.json` and set **Node.js Version** to `22.x` in **Vercel → Settings → General**.
6	**Contact form submits to the wrong WordPress site or localhost in production**	`PUBLIC_WORDPRESS_GRAPHQL_URL` was set in local `.env` but omitted in the Vercel Dashboard, or lacked the `PUBLIC_` prefix.	Add `PUBLIC_WORDPRESS_GRAPHQL_URL=https://wp.<domain.com>/graphql` in **Vercel → Settings → Environment Variables** and redeploy.
7	**Staging subdomain (`staging2.domain.com`) shows the cPanel "Default Web Site Page" or an SSL error**	Developer created the subdomain inside **cPanel → Domains** instead of **cPanel → Zone Editor**, creating a local A-record conflict.	Delete the subdomain from **cPanel → Domains**, then open **cPanel → Zone Editor** and add only the `CNAME` record pointing to Vercel's target.
8	**Google Analytics shows traffic from another client's website (cross-property pollution)**	`TrackingHead.astro` or `PUBLIC_GA_MEASUREMENT_ID` was left set to a previous client's GA4/Ads tag (as occurred with `AW-11145254205` vs `G-JP63DEKVVF`).	Create a dedicated GA4 Data Stream for the new client and set `PUBLIC_GA_MEASUREMENT_ID=G-XXXXXXXXXX` in both `.env` and Vercel before launch.

#	ERROR / SYMPTOM	ROOT CAUSE	EXACT FIX
9	**Only 10 or 100 blog posts appear on the site even though WordPress has 120+ posts**	WPGraphQL defaults to a 100-node ceiling per query (`graphql_connection_max_query_amount`).	Ensure the `graphql_connection_max_query_amount` filter returning `500` is active in `wp-content/mu-plugins/headless-cms-config.php` .
10	**Client-side scripts or GSAP animations stop firing after clicking a navigation link**	Using `window.addEventListener('DOMContentLoaded', ...)` instead of Astro's `<ClientRouter />` lifecycle event.	Wrap page initialization scripts in `document.addEventListener('astro:page-load', () => { ... })` so they run on both initial load and SPA navigation.

8 Handoff to Stage 2

In **Stage 2**, we will generate the automated CLI instruction file (`.md`) and scaffolding scripts that spin up a brand-new client marketing site from this template automatically. To prepare for Stage 2, the CLI automation file will require the following structured inputs:

1. **Client & brand identifiers**
   - `CLIENT_NAME` (e.g., `"Acme Heating & Cooling"`)
   - `REPO_SLUG` (e.g., `"acme-heating-frontend"`)
   - `PRIMARY_DOMAIN` (e.g., `"https://acmeheating.com"`)
   - `STAGING_DOMAIN` (e.g., `"https://staging2.acmeheating.com"`)
2. **WordPress headless origin configuration**
   - `WP_SUBDOMAIN_URL` (e.g., `"https://wp.acmeheating.com"`)
   - `WP_GRAPHQL_ENDPOINT` (e.g., `"https://wp.acmeheating.com/graphql"`)
   - `GRAVITY_FORM_ID` (e.g., `15` or `1`)
3. **Analytics & conversion identifiers**
   - `GA4_MEASUREMENT_ID` (e.g., `"G-XXXXXXXXXX"`)
   - `META_PIXEL_ID` *(optional, e.g., `"123456789012345"`)*
4. **Business schema & contact constants** (for `src/config/site.js` & JSON-LD `@graph`)
   - `COMPANY_PHONE_DISPLAY` & `COMPANY_PHONE_E164`
   - `COMPANY_EMAIL`
   - `POSTAL_ADDRESS` (`streetAddress`, `addressLocality`, `addressRegion`, `postalCode`, `addressCountry`)
5. **Service & case study route manifest**
   - List of initial `/services/[slug]/` pages and `/work/[slug]/` case studies to scaffold from our modular data-driven templates.